



Question 1 of 60

Flag

Scenario: Multi-Agent Research

In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

- ☐ **A** Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.
- ☒ **B** Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.
- ☐ **C** Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations.
- ☐ **D** Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization—the synthesis agent attempts web searches the report generator tries to analyze documents. What is the primary cause of this poor tool selection behavior?

- ☐ **A** The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.
- ☐ **B** Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.
- ☐ **C** The tool definitions consume too much context window space, leaving insufficient room for task content.
- ☒ **D** The agents' role descriptions in their system prompts conflict with having access to tools outside that role.

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

- ☐ **A** Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.
- ☐ **B** Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.
- ☒ **C** Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.
- ☐ **D** Pass the full accumulated context from all prior agents.

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

- ☒ **A** Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.
- ☐ **B** Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.
- ☐ **C** Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.
- ☐ **D** Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What is the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- ☐ **A** Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations
- ☒ **B** Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis
- ☐ **C** Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity
- ☐ **D** Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings, but it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

- ☐ **A** Standardize all subagent outputs to prose summaries with inline citations
- ☐ **B** Standardize all subagent outputs to JSON with fields for `claim`, `evidence`, `source`, and `confidence`
- ☒ **C** Update the synthesis agent to render each content type appropriately—financial data as tables, news as prose
- ☐ **D** Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation

Production monitoring shows that follow-up queries like "summarize what we learned about market trends" consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research. What's the most effective way to improve response time for these follow-up summaries?

- ☐ **A** Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand.
- ☒ **B** Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analysis.
- ☐ **C** Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings.
- ☐ **D** Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.

After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

- ☐ **A** The subagents need to share a single API connection to enable automatic context sharing between invocations.
- ☐ **B** The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.
- ☐ **C** The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.
- ☒ **D** The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps—for instance, noting “the retrieved sources discuss API authentication but lack details on token refresh patterns”—but under the current strict pipeline, this insight isn’t actionable since search has already completed. What’s the most effective architectural change?

- ☒ **A** Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.
- ☐ **B** Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.
- ☐ **C** Add a research planning agent before the search phase that decomposes topics into specific sub-questions.
- ☐ **D** Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate—it generates messages like “I’ll ask the web search agent to find sources on this topic”—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

- ☐ **A** Subagent context isolation means task descriptions from the coordinator don’t automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.
- ☒ **B** The coordinator’s allowedTools configuration doesn’t include “Task”, so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.
- ☐ **C** The AgentDefinitions are configured correctly, but the coordinator’s system prompt doesn’t explicitly list the available subagent types, preventing the model from knowing they can be invoked.
- ☐ **D** The coordinator’s max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information typically flow between these two subagents?

- ☐ **A** The web search agent directly invokes the document analysis agent, passing the discovered sources as parameters.
- ☐ **B** Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.
- ☐ **C** The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.
- ☒ **D** The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

The document analysis agent has a single `analyze_document` tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrow summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

- ☐ **A** Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent
- ☐ **B** Split the generic tool into purpose-specific tools—`extract_data_points`, `summarize_content`, `verify_claim_against_source`—each with defined input/output contracts
- ☒ **C** Keep the single tool but add an `analysis_type` enum parameter requiring explicit selection between extraction, summarization, and verification modes
- ☐ **D** Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats

When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2021: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such differences?

- ☐ **A** Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.
- ☒ **B** Require subagents to include publication or data collection dates in their structured outputs.
- ☐ **C** Configure the web search agent to only return results from the past 6 months.
- ☐ **D** Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.

The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent occasionally surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

- ☐ **A** Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category.
- ☐ **B** Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities.
- ☐ **C** Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure."
- ☒ **D** Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

- ☐ **A** Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.
- ☐ **B** Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.
- ☒ **C** Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.
- ☐ **D** Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.

The extraction pipeline receives documents of varying types—some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own JSON schema tailored to the document type. During testing, you observe that with `tool_choice: "auto"`, Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

- ☐ **A** Keep `tool_choice: "auto"` with system prompt instructions requiring tool use.
- ☐ **B** Set `tool_choice: "any"` with all extraction tools defined.
- ☒ **C** Consolidate all document types into a single unified-schema extraction tool and force that tool.
- ☐ **D** Add a preliminary classification call, then make a second call with `tool_choice` forced to the identified extraction tool.

Monitoring reveals that specifications sometimes appear inconsistently within source documents. For example, a summary section might state "Battery: 4000mAh" while the detailed specs table shows "Battery: 4200mAh". Your current schema uses a single `battery_capacity_mah` field. This inconsistency occurs in approximately 15% of documents, and historical analysis confirms the detailed specs table is more accurate 90% of the time. What's the most effective approach?

- ☐ **A** Include extraction instructions specifying to prefer values from the detailed specs table when multiple values exist, keeping the single-value schema.
- ☐ **B** Add a `conflict_detected` boolean field that flags inconsistencies, triggering manual review for affected documents.
- ☒ **C** Change to an array field that captures all values found with their source locations, letting downstream systems apply precedence rules.
- ☐ **D** Implement schema validation that rejects results containing conflicting values, requiring source document correction before processing.

Your extraction uses tool use with a JSON schema where `property_type` is defined as an enum: `['house', 'apartment', 'condo', 'townhouse']`. After deployment, 8% of extractions fail schema validation. A review of the data reveals listings mention many uncommon property types—"studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse"—and new types continue appearing regularly. What's the most effective solution?

- ☐ **A** Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.
- ☐ **B** Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.
- ☐ **C** Add an "other" value to your enum with a separate `property_type_detail` string field for specifics when "other" is selected.
- ☒ **D** Change `property_type` from an enum to a free-form string and implement a normalization step in post-processing.

Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts does not match the extracted grand total—sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals. What is the most effective approach to improve extraction reliability?

- ☐ **A** Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.
- ☐ **B** Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct.
- ☐ **C** Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.
- ☒ **D** Add a "calculated_total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ.

The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, negative, mixed). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than indicating this information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was... interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. What schema modification best addresses both issues?

- ☐ **A** Allow null values for pros/cons, and add "unclear" to the sentiment enum.
- ☐ **B** Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.
- ☒ **C** Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.
- ☐ **D** Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum.

Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- ☐ **A** Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.
- ☐ **B** Set `tool_choice` to `"any"` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.
- ☐ **C** Set `tool_choice` to `"auto"` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.
- ☒ **D** Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by custom_id. What's the most cost-effective approach to process these failures?

- ☒ **A** Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions
- ☐ **B** Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch
- ☐ **C** Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts
- ☐ **D** Resubmit the entire 10,000 document batch using a model tier with a larger context window

After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats—inline citations vs. bibliographies, methodology sections vs. details embedded in introductions. What's the most effective to address these failures?

- ☒ **A** Add few-shot examples demonstrating extractions from documents with varied structures—showing how to identify citations in different formats and locate methodology details across section types.
- ☐ **B** Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract.
- ☐ **C** Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.
- ☐ **D** Implement retry logic that re-sends requests when validation detects empty required fields.

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence $\geq 90\%$ have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

- ☐ **A** Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.
- ☒ **B** Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- ☐ **C** Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.
- ☐ **D** Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.

Your extraction pipeline occasionally receives responses that don't parse as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it. What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

- ☐ **A** Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.
- ☐ **B** Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.
- ☒ **C** Define a tool with a JSON schema specifying the expected structure, using tool_use to constrain Claude's output to schema-compliant JSON.
- ☐ **D** Implement a retry loop that catches JSON parse errors and re-prompts Claude with the error details, asking it to correct the malformed output.

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON. What approach provides the most reliable schema compliance?

- ☐ **A** Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.
- ☐ **B** Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.
- ☒ **C** Define a tool with your target schema as input parameters and have Claude call it with the extracted data.
- ☐ **D** Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails.

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned." What's the most effective improvement?

- ☐ **A** Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."
- ☒ **B** Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.
- ☐ **C** Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.
- ☐ **D** Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.

Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the "materials" field—sometimes returning "cotton blend", other times "Cotton/Polyester mix", and occasionally omitting the field when material information is clearly present in the source. What's the most effective way to improve extraction consistency?

- ☒ **A** Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats
- ☐ **B** Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability
- ☐ **C** Set temperature to 0 to eliminate randomness and ensure deterministic outputs
- ☐ **D** Make the "materials" field required instead of optional in the schema to force the model to always extract a value

The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures. What is the most reliable approach to ensure Claude's output consistently matches the schema?

- ☒ **A** Define a tool with an input schema matching your required JSON structure and extract the data from Claude's tool_use response.
- ☐ **B** Make two separate API calls—first extracting information as text, then asking Claude to format that text as JSON.
- ☐ **C** Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses.
- ☐ **D** Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON.

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions ($\geq 85\%$) also contain errors—cases where the model finds plausible but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time. What approach is most effective?

- ☐ **A** Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.
- ☒ **B** Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.
- ☐ **C** Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.
- ☐ **D** Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.

Your agent includes an `update_game_score` tool that accepts `game_date` (string), `home_team` (string), and `away_team` (string) parameters. Production logs reveal recurring issues: the agent uses team nicknames instead of official names, applies inconsistent date formats, and selects the wrong game when teams have rematches in the same season. What tool interface change would most effectively prevent these errors?

- ☒ **A** Replace the three parameters with a single `game_id` parameter and a separate `search_games` lookup tool that returns matching game IDs.
- ☐ **B** Add detailed examples to the tool description showing the required date format and complete list of official team names.
- ☐ **C** Add enum constraints listing valid team names for both team parameters, and add a regex pattern enforcing ISO 8601 format for the date parameter.
- ☐ **D** Add a `season` parameter to disambiguate rematches, and add a `confirm_before_update` flag that returns the resolved game details for the agent to verify before the score is committed.

Your `publish_article` tool calls an external CMS API that occasionally returns transient errors (network timeouts, 503s) and non-transient errors (403 permission denied, 422 validation failure). Currently, every error is returned directly to the agent, which leads to the agent retrying non-transient errors and wasting turns on failures that will never succeed. How should you partition error-handling responsibility between the tool implementation and the agent?

- ☐ **A** Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable—try again later" message, shielding the agent from error complexity.
- ☒ **B** Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation failures) to the agent with descriptive messages so it can take corrective action.
- ☐ **C** Handle all errors inside the tool: implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.
- ☐ **D** Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times—keeping the tool implementation stateless and simple.

Your `search_products` tool queries an external catalog API that returns paginated results (50 items per request). Production logs show queries frequently match 200+ products, and the current design that auto-fetches all pages causes 15-20 second delays. How should you redesign the pagination handling?

- ☐ **A** Create separate `search_products` and `fetch_more_results` tools for pagination.
- ☒ **B** Return the first page with total match count and cursor for additional pages.
- ☐ **C** Add a `max_pages` parameter (default: 2) that controls how many pages are fetched internally.
- ☐ **D** Implement server-side relevance ranking and return only the top 50 most relevant items.

Your resource allocation tool returns a simple acknowledgment message after provisioning is requested. Users frequently approve allocations and immediately ask "how much did that cost?" or "which project was that?" - indicating they confirmed without understanding the request. What tool design change would most effectively address this?

- ☐ **A** Add a `user_acknowledged: boolean` parameter that must be set true, with instructions for the agent to only set it after the user explicitly confirms they reviewed the details
- ☐ **B** Implement a 60-second hold before execution completes, allowing users time to review pending allocations and cancel if needed
- ☒ **C** Return structured data including cost estimate, target project, resource specifications, and impact summary in the tool response
- ☐ **D** Add a `detail_level` parameter with options "minimal" or "comprehensive" that controls how much context the agent presents in confirmations

Your `track_shipment(tracking_id)` tool queries an external logistics API that sometimes fails—the API may be temporarily unavailable, the tracking ID may be malformed, or the shipment may not exist. Currently, your tool raises a Python exception when errors occur. Users report the agent gives unhelpful responses like “I’m having trouble with that request” instead of suggesting alternatives such as verifying the tracking number format or checking by order number. How should you handle errors in tool results?

- ☐ **A** Implement retry logic with exponential backoff inside the tool implementation so transient errors are automatically handled and only return a result after all retry attempts are exhausted.
- ☒ **B** Return structured error information as normal tool output including error type, recoverability status, and actionable context for the user.
- ☐ **C** Create dedicated error-recovery tools (`retry_tracking_lookup` , `search_by_order_number`) that the model can invoke after the primary tracking tool returns a failure indicator.
- ☐ **D** Return a generic error response (e.g., `{"success": false, "error": "lookup_failed"}`) for all failure cases to maintain a consistent schema and avoid exposing internal error details.

Your document extraction tool uses ML models to extract invoice fields (vendor, amount, date). The models return confidence scores (0.0-1.0) for each extracted field. In production, you observe: (1) the agent proceeds with low-confidence extractions that are incorrect 23% of the time, and (2) the agent requests unnecessary human review for 31% of extractions that were actually correct. How should you restructure the tool's output?

- ☒ **A** Return fields with confidence scores, plus a `requires_review` boolean computed using your tested confidence thresholds, along with a `review_reasons` array explaining which fields triggered review.
- ☐ **B** Return fields organized into `verified` and `needs_verification` objects based on confidence thresholds.
- ☐ **C** Return fields with their raw confidence scores and add detailed few-shot examples to your system prompt demonstrating how to interpret different confidence ranges and when to request human review.
- ☐ **D** Compute an aggregate `extraction_quality` score across all fields and return it alongside the extracted values. Include a text summary describing the overall extraction reliability.

You're deciding whether to expose your weather API integration as an MCP server or implement it as a custom tool directly in your agent application. Your team plans to build several different AI applications that will all need weather data. What is the primary advantage of the MCP approach?

- ☐ **A** MCP servers process tool calls faster than custom implementations because they use an optimized binary protocol.
- ☒ **B** Any MCP-compatible client can connect to your weather server without writing custom integration code.
- ☐ **C** MCP automatically handles authentication and rate limiting with the weather API, reducing backend implementation work.
- ☐ **D** MCP provides built-in retry logic that automatically handles failed API calls with exponential backoff.

Your marketing agent connects to MCP servers from three different vendors: an email service, a social media platform, and an analytics dashboard. Each server provides tool annotations—some tools marked with `readOnlyHint: true`, others with `destructiveHint: true`. Your team proposes automatically bypassing confirmation prompts for tools annotated as read-only to improve workflow speed. What should guide this decision?

- ☐ **A** Read-only annotations indicate intended behavior and are reliable for tools that perform GET requests to external APIs. Skip confirmation for data-fetching tools from properly connected servers.
- ☐ **B** MCP servers run as local processes, so tools from any properly initialized connection inherit your application's security context and can be trusted regardless of vendor.
- ☐ **C** Implement a verification layer that tests each tool's actual behavior before adding it to an allowlist. Skip confirmation only for tools validated through behavioral testing.
- ☒ **D** Treat tool annotations as untrusted metadata unless the server itself is trusted. Base confirmation requirements on your assessment of each vendor's trustworthiness, not on their self-reported annotations.

Your `control_device` tool manages smart home devices through external APIs. When a device doesn't respond within the timeout period, the tool returns an error. Production logs show that the agent simply tells users "the device is not responding" without offering helpful next steps. Which error response structure would best enable the agent to provide useful follow-up?

- ☐ **A** Set `is_error: true` with a brief "Device offline" message and provide a separate tool the agent can call to retrieve context-specific troubleshooting suggestions.
- ☐ **B** Set `is_error: true` with a structured technical error containing the device ID, timeout duration, and raw API response code for debugging purposes.
- ☒ **C** Set `is_error: true` with a message explaining the likely cause and suggesting troubleshooting steps the agent can offer the user.
- ☐ **D** Set `is_error: false` with an optimistic message indicating the command was dispatched successfully but device acknowledgment is still pending.

Your `get_portfolio_value` tool returns the total value of a user's investment portfolio. You're deciding between returning a structured JSON object with explicit fields versus returning the information as a formatted text string. What is the primary advantage of using structured output with defined fields?

- ☐ **A** Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs.
- ☐ **B** JSON schemas automatically validate that the underlying API returned correct data before the agent processes it.
- ☒ **C** The agent can reliably extract specific values without parsing free-form text, reducing errors in subsequent operations.
- ☐ **D** Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values.

Your content curation agent discovers articles, analyzes each for relevance, then adds selected articles to themed collections. With separate `discover_articles(topic)`, `analyze_article(id)`, and `add_to_collection(article_id, collection_id)` tools, you observe 18+ sequential tool calls per request, causing latency issues. The agent must make editorial judgments about which articles fit a collection's theme—this requires seeing all candidates with their analysis scores simultaneously to select a cohesive set. What tool composition best addresses efficiency while preserving editorial judgment?

- ☐ **A** Keep all tools separate but implement response caching for `analyze_article` calls.
- ☐ **B** Create a `curate_collection(topic, collection_id)` tool that handles discovery, analysis, and selection internally using configurable quality thresholds.
- ☐ **C** Add a `preview_curation(topic, collection_id)` tool that shows what would be added based on predefined rules, with an `approve_curation()` tool to confirm.
- ☒ **D** Create a `discover_and_analyze(topic)` composite tool that returns all candidates with their analysis scores, keeping `add_to_collection` separate for selective calls.

Your shipment tracking tool queries multiple carriers (FedEx, UPS, DHL) that each return status information in different formats—FedEx uses numeric codes, UPS uses descriptive phrases, and DHL returns timestamped event arrays. The agent uses these results to determine delivery status and escalate delayed shipments. How should you structure the tool's return value to best enable the agent's reasoning?

- ☐ **A** Return raw carrier responses with source metadata, encoding carrier-specific interpretation logic in the system prompt.
- ☐ **B** Return both a normalized summary and the complete raw carrier response in each tool call.
- ☒ **C** Return a normalized schema with `status`, `estimated_delivery`, `delay_reason`, and `requires_action` fields, converting carrier formats internally.
- ☐ **D** Design separate tool endpoints for each carrier (`track_fedex`, `track_ups`, `track_dhl`) with carrier-specific response schemas.

Your `search_flights` tool calls an external airline API that occasionally returns a 503 Service Unavailable error. What is the most effective way to handle this error in your tool implementation?

- ☐ **A** Log the error internally and return an empty response, letting the model continue without the flight data.
- ☐ **B** Return an empty flight list as if the search succeeded but found no matching flights.
- ☒ **C** Automatically retry the request up to five times with exponential backoff before returning results to the agent.
- ☐ **D** Return an error message in the tool result explaining the service is temporarily unavailable.

Your `update_user_profile` tool accepts a `user_id` (required) and an optional `fields_to_update` object. In testing, Claude frequently omits `user_id` or passes incorrectly structured data. What is most critical for helping Claude understand what parameter values to provide?

- ☐ **A** Clear parameter descriptions explaining expected format, such as "`user_id`: UUID of the user to update (required)"
- ☒ **B** Strict JSON Schema type constraints marking `user_id` as required and defining `fields_to_update` as an object type
- ☐ **C** Detailed error responses explaining why invalid parameter values were rejected
- ☐ **D** Verbose parameter names encoding format hints, such as `user_id_string_uuid_format`

Your MCP server implements a `check_availability` tool that queries an external calendar API. During testing, you encounter three error conditions: (1) the tool is called with a malformed request missing the required `user_email` parameter, (2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system, and (3) the calendar API returns a 503 because the service is temporarily unavailable. How should each error be reported according to MCP's error handling design?

- ☐ **A** Report error 1 as a JSON-RPC protocol error; report errors 2 and 3 as tool results with `isError: true`.
- ☒ **B** Report errors 1 and 2 as JSON-RPC protocol errors; report error 3 as a tool result with `isError: true`.
- ☐ **C** Report all three as tool results with `isError: true`.
- ☐ **D** Report all three as JSON-RPC protocol errors.

Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order`, it identifies that the dispute involves a promotional pricing error requiring manager approval—beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

- ☐ **A** Call `escalate_to_human` passing only the customer's original message.
- ☐ **B** Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.
- ☒ **C** Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`.
- ☐ **D** Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.

After investigating a billing dispute over 25+ turns, you've identified that duplicate charges occurred due to a payment gateway timeout triggering retry logic. The required refund (\$847) exceeds your \$500 authorization limit. You need to call `escalate_to_human`, and the human agent won't have access to your conversation transcript. What context should you pass to enable effective resolution?

- ☒ **A** A structured summary: customer ID, root cause, refund amount, and recommended action.
- ☐ **B** The complete conversation transcript with all tool results.
- ☐ **C** Your diagnosis and the refund amount only.
- ☐ **D** The customer's original complaint verbatim plus the tool result excerpts showing duplicate transactions.

Your agent has called `lookup_order` multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history). Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before making additional lookups?

- ☐ **A** Proceed with additional lookups without modifying the existing tool output context
- ☐ **B** Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues
- ☒ **C** Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details
- ☐ **D** Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions

A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

- ☒ **A** Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.
- ☐ **B** Implement sliding window context that retains the most recent 30 turns.
- ☐ **C** Summarize earlier turns into a narrative description, preserving full message history only for the active issue.
- ☐ **D** Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.

Production logs reveal inconsistent error handling: when `lookup_order` fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: `{"isError": true, "content": [{"type": "text", "text": "Operation failed"}]}`. The agent cannot distinguish between error types. What's the most effective improvement?

- ☐ **A** Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.
- ☒ **B** Enhance error responses with structured metadata: include `errorCategory` (transient/validation/permission), `isRetryable` boolean, and a description of what caused the failure.
- ☐ **C** Create an `analyze_error` MCP tool the agent calls after any failure to determine the error category and recommended action.
- ☐ **D** Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.

When the agent calls `lookup_order` and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?

- ☐ **A** The orchestration layer automatically routes to the next tool based on the order's status field.
- ☐ **B** The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.
- ☐ **C** The agent executes the remaining steps in a tool sequence planned at the start of the request.
- ☒ **D** The order details are added to the conversation and the model reasons about which action to take.

A customer sends: "This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW." The agent has not yet called any tools to investigate their account. What should the agent do?

- ☒ **A** Acknowledge the frustration and ask one targeted question to understand the specific issue before escalating.
- ☐ **B** First call `get_customer` and `lookup_order` to gather account context, then escalate to a human agent.
- ☐ **C** Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats their request.
- ☐ **D** Immediately call `escalate_to_human` with the conversation history.

You're implementing the escalation logic for when the agent should call `escalate_to_human`. Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

- ☒ **A** Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.
- ☐ **B** Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.
- ☐ **C** Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.
- ☐ **D** Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

- ☒ **A** The conversation history isn't being passed in subsequent API requests.
- ☐ **B** The prompt lacks instructions telling Claude to remember information across multiple exchanges.
- ☐ **C** Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.
- ☐ **D** The verification tool is clearing the agent's internal state after each successful validation step.

During a billing dispute resolution, your agent successfully retrieves customer info via `get_customer` and order details via `lookup_order`, but when attempting to call `process_refund`, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

- ☐ **A** Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically
- ☐ **B** Implement automatic retries with exponential backoff for `process_refund`, keeping the conversation open until the refund is successfully processed
- ☒ **C** Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later
- ☐ **D** Escalate immediately to a human agent since the refund action cannot be completed

When implementing your `lookup_order` MCP tool, the backend sometimes returns errors (e.g., "Order not found" or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

- ☐ **A** Return a success response with a "status" field indicating the error type
- ☐ **B** Log the error server-side and return an empty result to avoid confusing the model
- ☐ **C** Throw an exception from the tool handler so the agent framework can catch and log it
- ☒ **D** Return the error message in the tool result content with the `isError` flag set to true

A customer writes: "I've been going back and forth on this return for days. I just want to speak to someone who can actually help me." The agent has confirmed via `lookup_order` that the return is straightforward—within policy and eligible for immediate processing. What should the agent do?

- ☐ **A** Call `escalate_to_human` immediately to honor the customer's request
- ☒ **B** Acknowledge frustration, inform them this is resolvable now, and offer to complete it or escalate
- ☐ **C** Ask what specifically hasn't worked in previous attempts before deciding whether to escalate or resolve automatically
- ☐ **D** Process the refund via `process_refund` to resolve the underlying issue, then inform them it's complete

Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent—this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

- ☐ **A** Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).
- ☐ **B** Strengthen the system prompt with emphatic language: "CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly."
- ☒ **C** Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation.
- ☐ **D** Modify the refund tool to return an error with message "Amount exceeds policy limit—please escalate" when threshold is exceeded.

A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains `lookup_order` results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information. What approach most reliably handles returning customers?

- ☐ **A** Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.
- ☐ **B** Resume with full history but filter out previous `tool_result` messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.
- ☒ **C** Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.
- ☐ **D** Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness.

Your `process_refund` tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (5% of calls), and business errors ("Order exceeds 30-day return window", "Item already refunded") that are permanent (12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude. What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- ☐ **A** Add a `check_refund_eligibility` tool that must be called before `process_refund` to prevent business rule violations.
- ☒ **B** Return structured error responses with `retriable: false` for business errors and a customer-friendly explanation for Claude to use.
- ☐ **C** Add few-shot examples showing how to distinguish retrieable from non-retrieable errors by parsing error message text.
- ☐ **D** Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.